

Programming Instructions

For the Endeavor Humanoid by ProtoDane Robotics

Requirements

- VSCode with PlatformIO
- (Optional) VS Code + Arm GNU Toolchain (mingw32) for working with the Servo2040; binary files and the communication protocol have been provided.

1: Flash the Servo 2040

Unless you want to add custom functionality, the binary files for the Servo 2040 have been provided to make the Pico SDK installs optional (this document does not cover the Servo2040 source code).

1. Download the binary file from the GitHub repository under servo2040/firmware/binaries. If using the Servo2040 without IMU, download servo2040_client.uf2. Otherwise, use servo2040_client_imu.uf2.
2. Plug in the Servo 2040 board to a computer via USB-C cable while holding down the BOOT button. Within a few seconds a file storage for the 2040 should pop up on the desktop.
3. Drag the .uf2 file into the 2040's file storage. The file storage should disappear and LED lights should start coming up.

2: Configure the ESP32 code and upload

1. Open firmware/esp32_controller in VSCode and wait for PlatformIO to initialize and configure the project.
2. Find the MAC address of the Bluetooth controller you want to use for your robot and set that as the value of CTRL_MAC.

- a. The MAC address of the Bluetooth controller can be found by running the 'Controller' sketch within the Bluepad32 Examples folder.
- b. Upload the sketch to the ESP32 (or a spare one if you have it). Open the serial terminal (9600 baud rate).
- c. Set your controller to pairing mode and wait for the connection message with the MAC address in the terminal.

```
CALLBACK: Controller is connected, index=0  
BTAddr: 98:b6:d5:20:65:22, VID/PID: 057e:2009, flags: 0x03  
idx=0, addr=98:b6:d5:20:65:22, vid=0x057e, pid=0x2009, name=ESP32, type=0, pin=0
```

3. Open platformio.ini and scroll down to [env:my_robot]. This is where you will configure your robot parameters; if you have multiple robots or different configurations, you should copy the lines including and below line 20.
 - a. **CTRL_MAC:** MAC Address of Bluetooth controller
 - b. **IMU_CONFIG:** 0 = IMU Disabled | 1 = BNO-055 | 2 = BNO-08X
 - c. **SERIAL_DEBUG_MODE:** If set true, disables all motor functions to safely troubleshoot over USB cable
 - d. **FALL_PROTECTION_ENABLED:** If set true, relay will automatically switch off when IMU_TIP_THRESHOLD (config.h) is reached
 - e. **ULT_LOCK:** If set true, will disable the tumble "ultimate" move for safety
 - f. **CONFIG_FILE:** If uncommented, will use the specified file name instead of robotInfoDefault.h
4. Navigate to robotInfoDefault.h (or your copied file). Fill in the servoStruct definitions with the servo calibration values that you recorded. The formatting is as follows: {min, mid, max, is180}, where is180 is a boolean value that is true of the servo is a 180-degree servo.
 - a. Example:

```

const servoStruct s_torso = {515, 1458, 2397, false}; // TORSO
const servoStruct s_ra1 = {510, 1495, 2397, false}; // RIGHT ARM SHOULDER BASE
const servoStruct s_ra2 = {543, 1505, 2413, false}; // RIGHT ARM SHOULDER SWING
const servoStruct s_ra3 = {592, 1495, 2404, true}; // RIGHT ARM BICEP SWIVEL
const servoStruct s_ra4 = {593, 1493, 2385, true}; // RIGHT ARM ELBOW
const servoStruct s_rl1 = {592, 1512, 2405, true}; // RIGHT LEG THIGH
const servoStruct s_rl2 = {588, 1490, 2372, true}; // RIGHT UPPER LEG
const servoStruct s_rl3 = {643, 1550, 2432, true}; // RIGHT LOWER LEG
const servoStruct s_rl4 = {587, 1493, 2405, true}; // RIGHT LEG ANKLE
const servoStruct s_la1 = {505, 1457, 2409, false}; // LEFT ARM SHOULDER BASE
const servoStruct s_la2 = {550, 1509, 2453, false}; // LEFT ARM SHOULDER SWING
const servoStruct s_la3 = {622, 1530, 2480, true}; // LEFT ARM BICEP SWIVEL
const servoStruct s_la4 = {622, 1533, 2440, true}; // LEFT ARM ELBOW
const servoStruct s_ll1 = {597, 1500, 2410, true}; // LEFT LEG THIGH
const servoStruct s_ll2 = {685, 1580, 2470, true}; // LEFT UPPER LEG
const servoStruct s_ll3 = {567, 1522, 2437, true}; // LEFT LOWER LEG
const servoStruct s_ll4 = {591, 1520, 2428, true}; // LEFT LEG ANKLE

```

Note, you shouldn't need to change the servoCluster mapping unless your servo wiring is different from the Wiring doc.

5. Plug in the ESP32 to your computer. Compile the project and upload.

Sanity Checklist

- ☐ After a few seconds the LED bar on the Servo2040 flashes green
- ☐ After a few seconds the LEDs inside the robot's head flashes
- ☐ Pairing the Bluetooth controller to the robot is successful
- ☐ Pressing the Home Button brings the robot to its idle position

Appendix A: Servo2040 Communication Protocol

- Servo2040 communicates with the ESP32 over UART1 at a baud rate of 115200
- Upon startup, the LED bar will light up to signal certain stages of the initialization process. The LED bar will flash all green to indicate that it is ready for operation.
 - **The Servo2040 will yield until the ESP32 sends a handshake message of 0b01101001 or a hexadecimal of 0x69 (this was not intentional, I swear).**
- The Servo2040 waits for the ESP32 to receive a command byte, in which the leading 4 bits are known as the preamble; each value used to indicate what to send back to the

ESP32, however this feature has been deprecated. The Servo2040 will recognize the following bytes:

- 0x08
- 0x09
- 0x0F
- The trailing 4 bits of the command bit is the command code, and each value indicates an action the Servo2040 executes:
 - **0x0F | CMD_ENABLE:** Allows transmission of PWM signals to the servos.
 - **0x01 | CMD_DISABLE:** Discontinues sending PWM signals to the servos.
 - **0x02 | CMD_LED:** Controls the LED bar
 - The following byte controls which LEDs are lit. For instance, if the byte value is 0b10011, this indicates that LEDs 1, 2, and 5 are lit.
 - Example full message: (0x0F << 4 | CMD_LED) (0b10011)
 - **0x07 | CMD_PULSE:** Sends PWM signals to all servos
 - The command byte is followed by 36 bytes, each pair of bytes representing the target PWM pulse for a specific servo for a total of 18 actuators.
 - The pulse values must be between 500 and 2500 per manufacturing specifications. If the pulse value falls outside this range the servo will be ignored; this is implemented such that the user can control which servos need to be actuated.
 - Example message: (0x0F << 4 | CMD_PULSE) (10) (50) (0) (0) (15) (0) (15) (0) (15) (0) [...]
 - Sets SERVO1 to 1050, ignores SERVO2, SERVO3-SERVO18 to 1500
 - **0x06 | CMD_PULSE_DELAY:** Moves all servos from current position to target position within a specified time delay.
 - The command byte is followed by two bytes indicating the time delay in milliseconds, then the 36 bytes for the 18 servos.
 - Example message: (0x0F << 4 | CMD_PULSE_DELAY) (20) (50) [Same parameters as CMD_PULSE]
 - Moves all servos to desired position after 2050 ms
 - *Note: speed of servos depend on the most recent pulse or delayed pulse command*